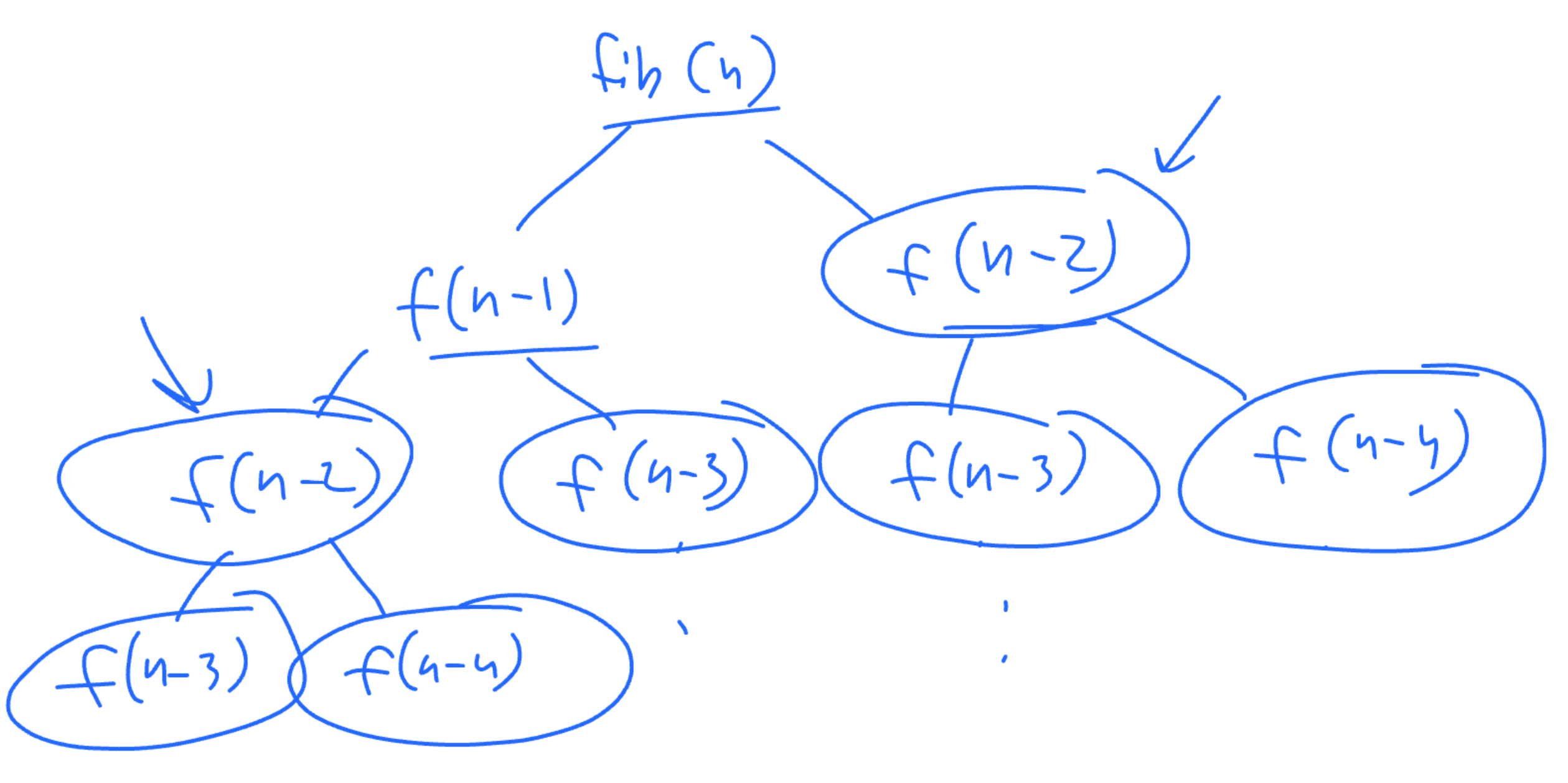
DP

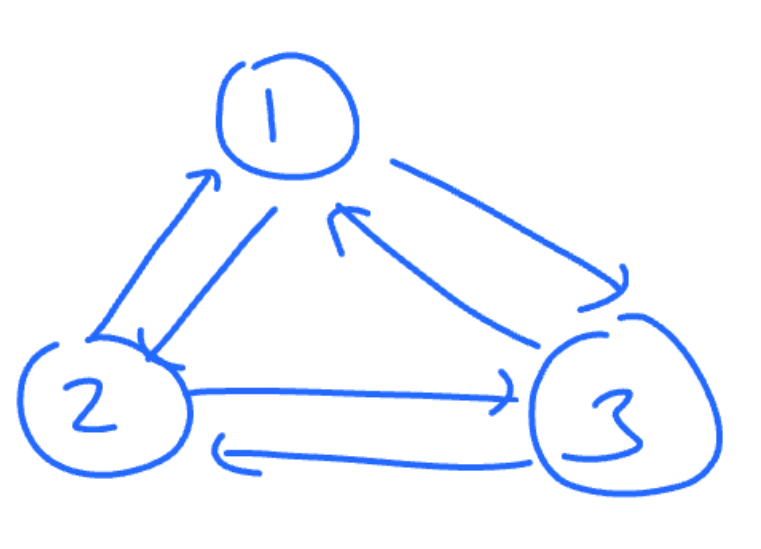
Keep on storing the results for future references.

2 essential conditions:

1) Overlapping Subproblems.



2) Optimal | correct soln. to the subproblems must lead to the optimal | correct soln. of the overall problem.



Longest Dist (LD)

$LD(1,2) = 3$

$LD(2,3) = 3$

$$\underset{3}{\underline{LD(1,3)}} \neq \underset{3}{\underline{LD(1,2)}} + \underset{3}{\underline{LD(2,3)}}$$

Top-Down DP (Memoization)

- Recursive in nature.

Bottom Up DP (Tabulation).

- Iterative in nature.

0 1 2 3 4 - - . - . 100.

LCS

LCS(~~AB~~C,

LCS(ABCDGH, AEDFHR)

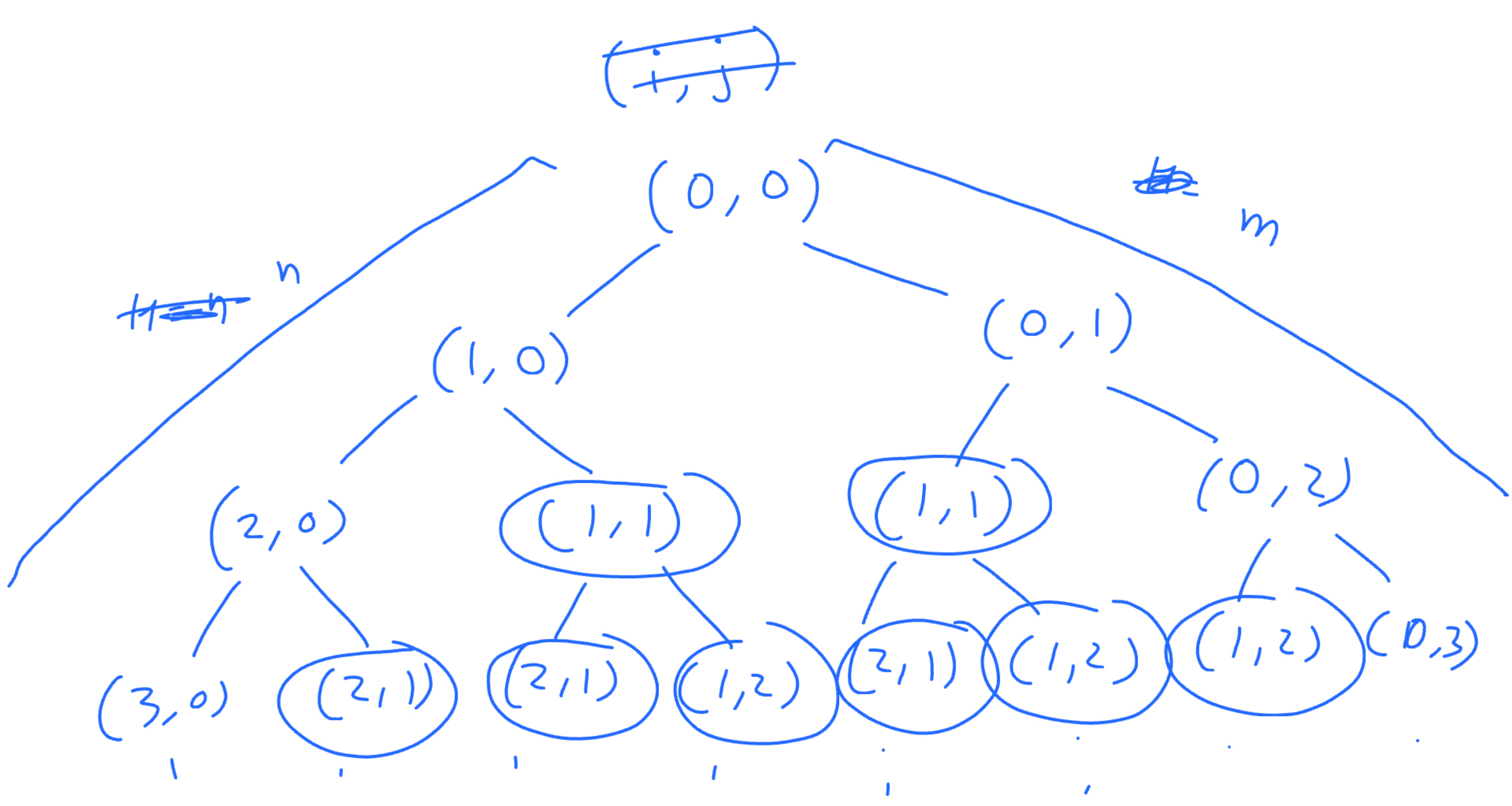
$\downarrow$

1+ LCS (BCDGH, EDFHR)

$\downarrow$

max (LCS(CDGH, EDFHR), LCS(BCDGH, DFHR))

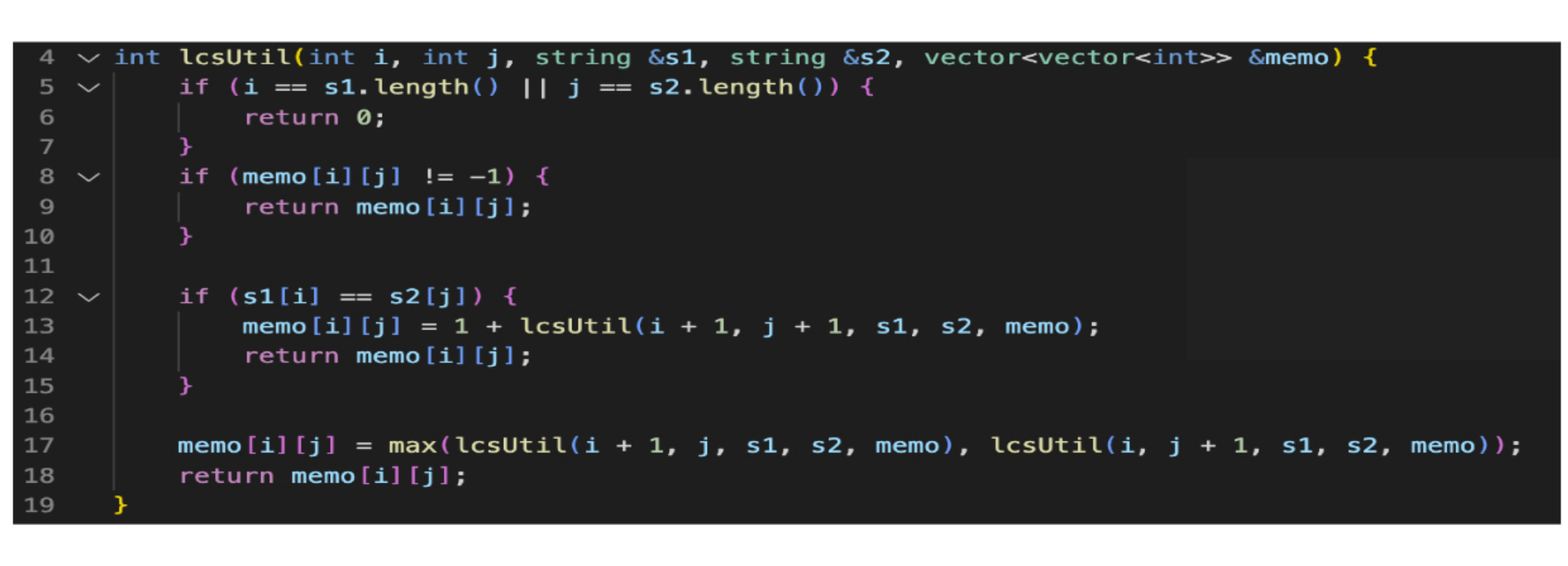
$$LCS(i,j) = \begin{cases} 1 + LCS(i+1, j+1) & \text{if } s1[i] == s2[j]. \\ \max(LCS(i+1, j), LCS(i, j+1)) & \text{otherwise.} \end{cases}$$



H = max(n, m)

$TC = O(2^{\max(n,m)})$

AS = max(n, m)



```
int lcsUtil(int i, int j, string &s1, string &s2, vector<vector<int>> &memo) {
    if (i == s1.length() || j == s2.length()) {
        return 0;
    }
    if (memo[i][j] != -1) {
        return memo[i][j];
    }

    if (s1[i] == s2[j]) {
        memo[i][j] = 1 + lcsUtil(i + 1, j + 1, s1, s2, memo);
        return memo[i][j];
    }

    memo[i][j] = max(lcsUtil(i + 1, j, s1, s2, memo), lcsUtil(i, j + 1, s1, s2, memo));
    return memo[i][j];
}
```